

1. AM-D02 机械臂运动学算法开发与 Sim2Real

本学期，我作为重庆大学具身智能实验室机械臂运动学负责人，主导了其自主研发的 AM-D02 七自由度冗余机械臂及 AM-DP 半人形机器人的机载运动学控制系统构建。期间，我通过提纯 KDL 核心算法，独立搭建了完整的运动学解算库，并结合动力学约束在 STM32 平台上实现了机械臂的平滑轨迹规划，完成了包括硬件在环仿真、单臂、半人形机器人整体控制等工作，成功实现了 Sim2Real 的完整项目开发流程。

1.1. 项目背景与实验平台

1.1.1. 开发意义

在具身智能与仿生机器人技术快速发展的背景下，多自由度冗余机械臂作为机器人与物理世界交互的核心操作末端，对其控制算法的实时性、平稳性及精确度提出了极高的要求。相比于传统的六自由度机械臂，七自由度机械臂通过引入一个冗余自由度，构成了与人类手臂高度相似的运动学构型。这种冗余性使得机械臂能够在保持末端位姿完全不变的前提下，利用零空间实现避障、避奇异及关节转矩优化等高级任务。

然而，传统的机器人运动学开源库（如 Orocos KDL）通常运行于高性能上位机（ROS/Linux 环境）中，其内部充斥着大量的动态内存分配与复杂的 C++ 容器操作，无法直接部署于资源受限的嵌入式微控制器（MCU）中。

为了打破上位机控制的局限，实现真正的机载全自主全动力学控制，本项目旨在提纯并重构 KDL 核心算法，搭建一套具备全平滑轨迹规划能力的轻量级嵌入式运动学解算器。项目依次经历“KDL 算法提纯→RViz 仿真验证→AM-D02 单臂物理适配→AM-DP 半人形双臂并行解耦”的完整演进，最终打通了从仿真到实体硬件（Sim2Real）的完整闭环。

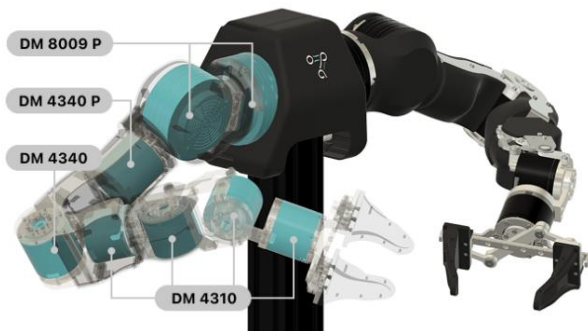


图 1.1.1.1 OpenArm 基本构型



图 1.1.1.2 AM-DP 渲染图

1.1.2. 实验平台概述

① OpenArm 七自由度冗余机械臂

OpenArm 是典型的 S-R-S 冗余机械臂，其作为一款安全、开源、高性能的人形机械臂平台，为物理 AI 研究和工业协作提供了理想解决方案。它在保证安全性的同时，提供了类人操作性能和强大的开发灵活性，使研究人员和开发者能够快速迭代和部署创新应用。

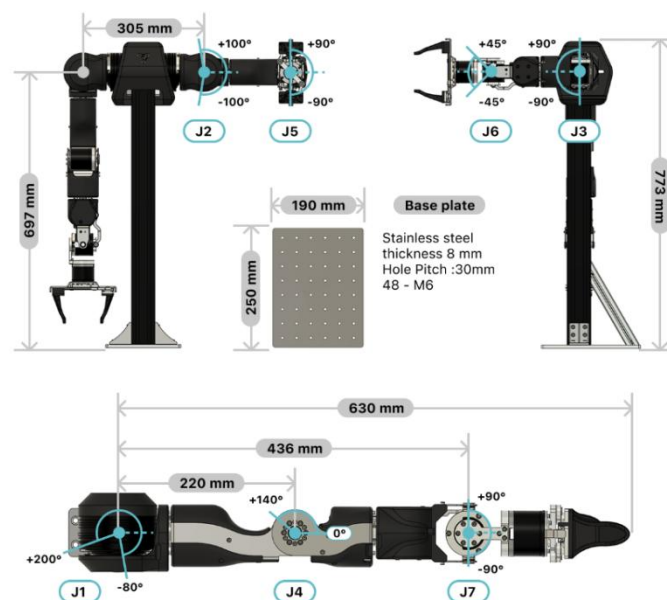


图 1. 1. 2. 1 OpenArm 硬件参数

② AM-D02 七自由度冗余机械臂：

AM-D02 是实验室自主研发的一款高性能串联旋转关节冗余机械臂。其物理构型完美映射了人类手臂的生物学关节分布：

- **肩部复合关节：**包含 3 个相互垂直的偏置自由度（Pitch-Yaw-Roll），负责大范围的空间定位。
- **肘部俯仰关节：**包含 1 个自由度（Pitch），提供核心的伸缩与弯曲能力。
- **腕部奇异姿态关节：**包含 3 个高集成度的交汇自由度（Yaw-Pitch-Roll），用于精细的末端姿态调整。

AM-D02 内部搭载了高精度绝对值编码器与高性能伺服电机驱动器，其连杆长度、关节偏置及物理几何极限均已沉淀为标准的 URDF 配置文件。

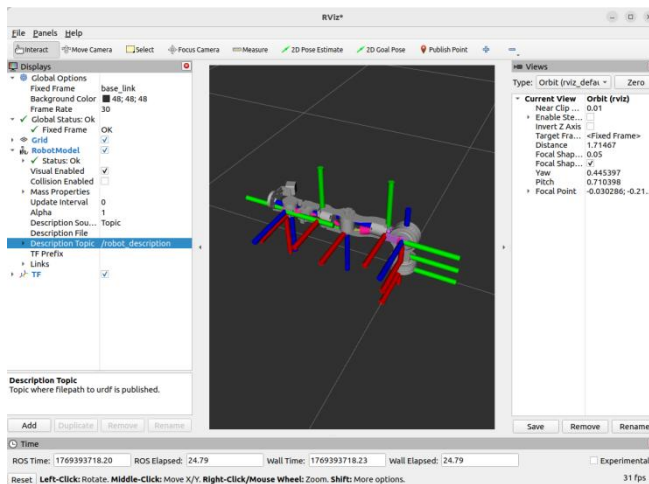


图 1.1.2.2 AM-D02 可视化

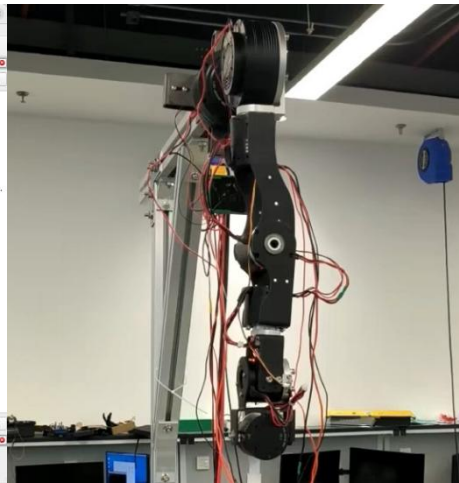


图 1.1.2.3 AM-D02 实物

③ AM-DP 半人形机器人：

AM-DP 是实验室面向复杂具身双臂操作任务研发的半人形机器人平台。该平台高度集成了两台对称安装的 AM-D02 冗余机械臂作为其上肢操作器，中部为仿生躯干及双目视觉头部感知系统，底部采用全向移动底盘。AM-DP 的双臂底座在物理空间上相对于机器人几何中心（Base Link）存在对称的安装偏置与旋转翻转。这一构型要求底层解算器不仅能进行单臂的高效求解，还必须具备双臂运动学解耦与中央变换补偿的能力。

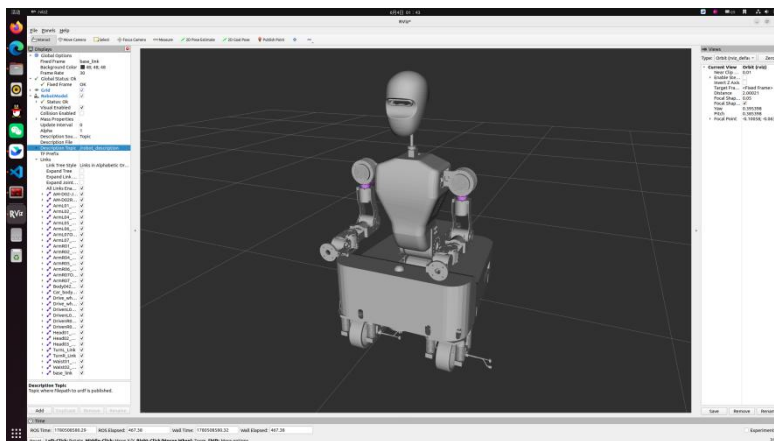


图 1.1.2.4 AM-DP 可视化

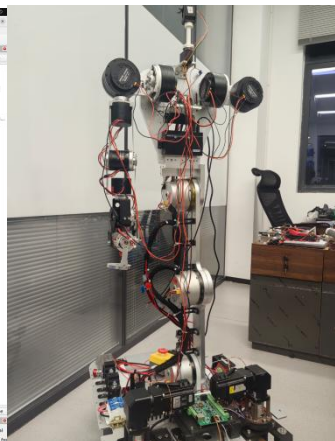


图 1.1.2.5 AM-DP 实物

1.2. 仿真环境搭建与 KDL 提纯：

在算法正式部署到真实物理硬件之前，必须在软件层面上完成开发环境的搭建以及核心算法的数学验证。

1.2.1. 仿真环境拓扑搭建

我使用的系统是 Linux 的 Ubuntu22.04, 仿真环境基于 ROS2, 编译器为 VScode。此外，我选择 Rviz 作为可视化机器人模型的平台。

我通过编写和导入机械臂的描述文件（URDF），将机械臂的物理连杆尺寸、关节限位、传动轴向等信息完全数字化。仿真环境的拓扑核心在于构建一个高保真 RViz 可视化沙盒。上位机通过监听串口下发的关节状态包，实时更新机器人状态发布器，从而在虚拟空间中 100% 重现机械臂的空间姿态，这为检测逆运动学解算的收敛性与轨迹连续性提供了直观的评估手段。

整体的仿真环境搭建流程如下图所示：

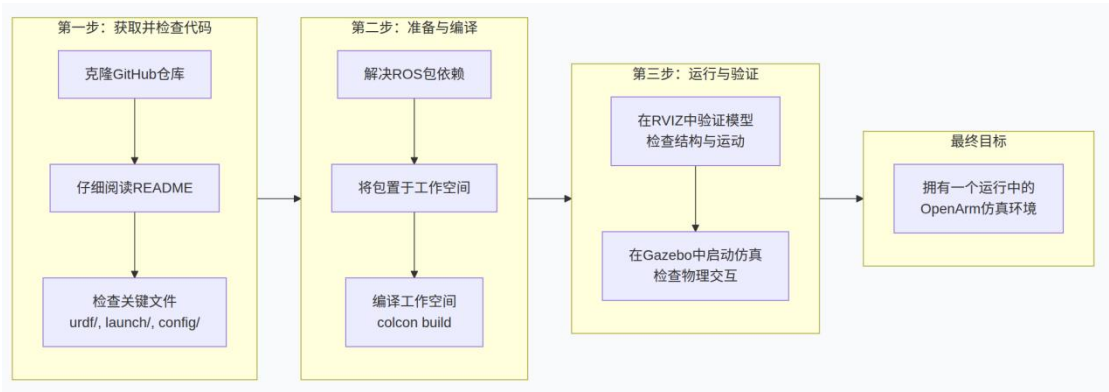


图 1. 2. 1. 1 仿真环境搭建流程

我调用了 openarm_description 仓库中的 URDF 模型，通过配置启动文件将其可视化在 Rviz 中。为了验证机械臂的运动链路是否完整，我使用图形化 API——GUI 滑块对机械臂的七个关节角（Openarm_joint1~7）进行控制。

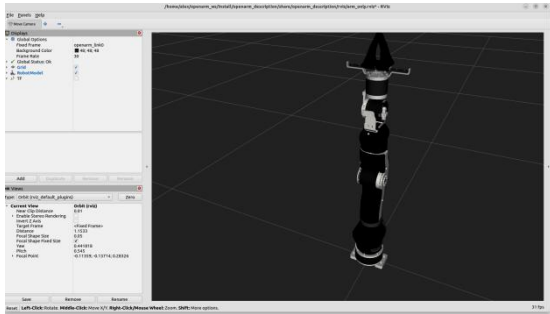


图 1. 2. 1. 2 Rviz 平台可视化

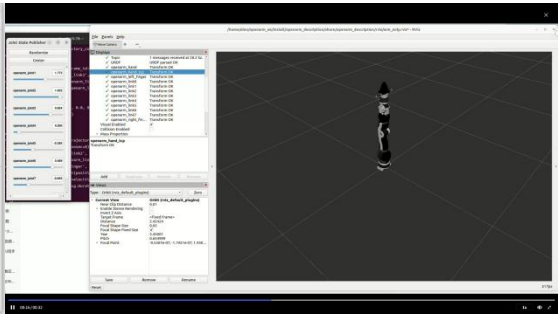


图 1. 2. 1. 3 GUI 滑块控制

1. 2. 2. KDL 库的算法提纯与重构：

Orocos KDL（Kinematics and Dynamics Library）是通用机器人学中极具权威性的开源解算库，其正向运动学(FK)与基于雅可比矩阵迭代的逆运动学(IK)算法具有极高的数据鲁棒性。其作为一个轻量级、跨平台的开源 C++ 库，专为关节型机器人（如机械臂）设计，是机器人开发中解决“运动规划与控制”核心问题的基础工具库，也是 ROS/ROS2 生态中运动学模块的核心依赖。

然而，KDL 的原生架构是专为 Linux/X86 平台设计的，直接移植到 STM32 单片机上会遭遇严重的动力学实时性崩溃。因此，在验证了 KDL 库后，我对其进行解耦与提纯，重新搭建了一个专用于单片机的运动学框架。

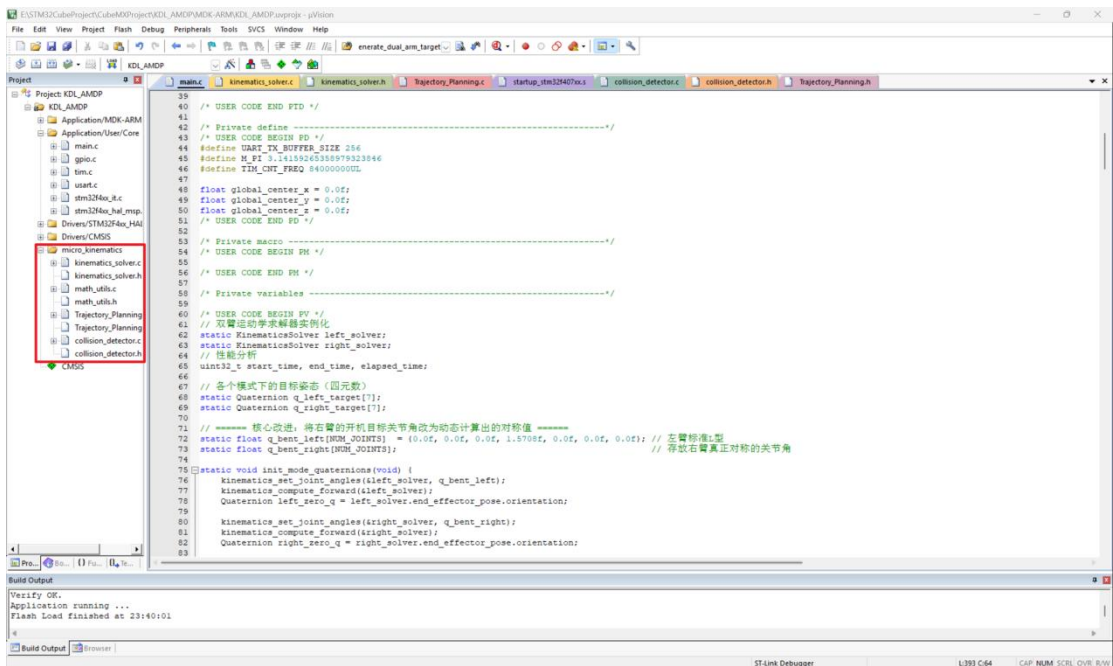


图 1.2.2.1 重构的 C 语言运动学库

如图 1.2.2.1,核心运动学文件为 kinematics_solver.c/h,math_utils.c/h, Trajectory_Planning.c/h, 分别对应运动学求解器的构建、运动学解算所需的数学函数与工具、轨迹规划任务,总代码数约 1500 行。我使用结构体的形式将核心运动学函数封装在其中,便于调用,可移植性高。

1.2.3. 单片机移植:

基于重新构建的运动学库,我进行了单片机的算法移植。我使用正点原子探索者开发板,其 MCU 为 STM32F407ZGT6,具有 FPU 浮点运算单元(一种集成在 MCU 中的硬件专用模块,专门用于加速浮点数运算操作)和足够的 RAM (192KB),可以高效处理浮点运算,且拥有丰富的各类接口,可以极大程度方便后续的调试。

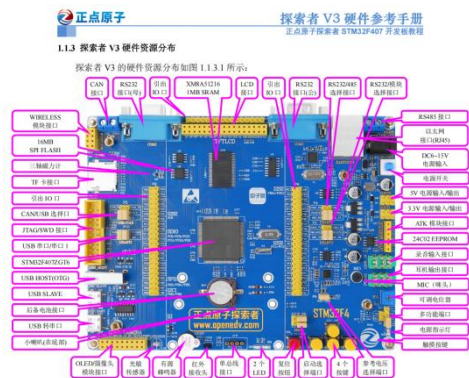


图 1.2.3.1 探索 V3 硬件资源分布

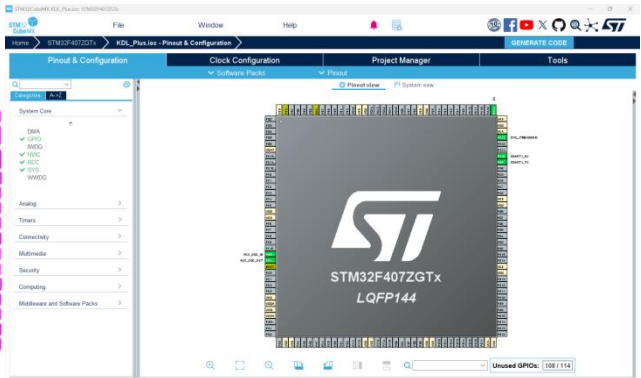


图 1.2.3.2 引脚资源与配置

我使用的工具链是 STM32CubeMX+Keilv5，是基于 HAL 库的 STM32 代码框架。为提高计算效率，选择使用外部高速晶振，将主频提升至 168MHz，并启用 FPU 浮点计算单元。开启 USART1，使用 DMA 传输的方式将计算结果通过 USBtoTTL 模块上传至 ROS 端。

在开始硬件在环仿真前，我首先对比了原生 KDL 库与单片机解算出的运动学数据，对单片机中的核心运动学代码进行验证，从而避免 ROS 虚拟环境迁移到嵌入式硬件中可能产生的错误，这也是现实工程中一个不可或缺的重要流程。

我在 Keil 与 ROS 中给定一致的机械臂 DH 参数和关节角指令，并用串口捕获输出的解算结果，如下图所示：

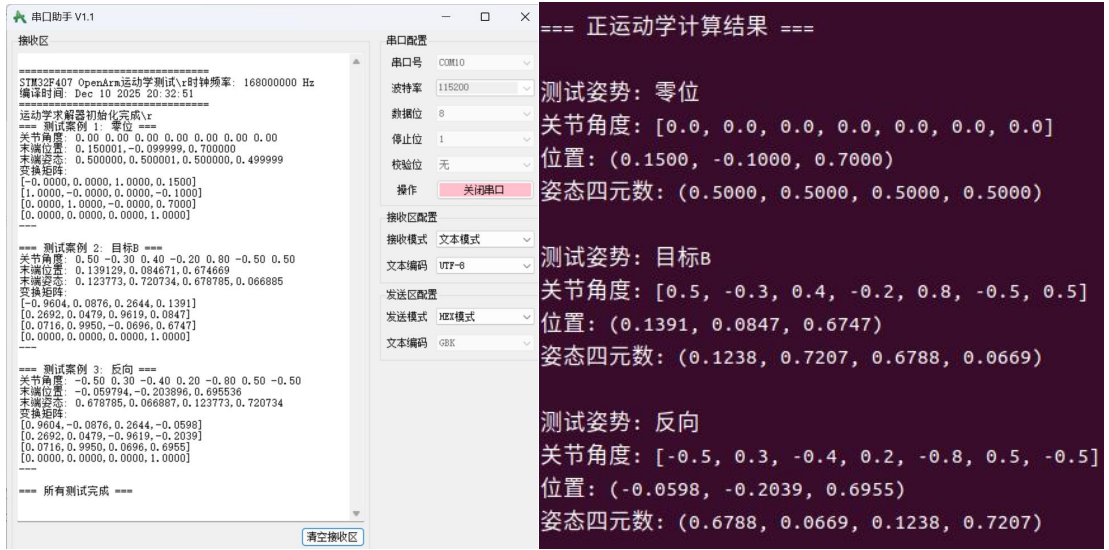


图 1.2.3.3 重构库的解算结果（STM32 端）

图 1.2.2 KDL 库解算结果（ROS 端）

通过与 ROS 端 KDL 库计算得出的结果相对比，可以发现两种方法算出的机械臂末端位姿数据完全一致，由此验证了重构库的核心运动学算法的正确性。

为了让 ROS 端能接收到 STM32 发送的数据，我创建了一个 Rviz 专用的串口节点脚本 scripts/rviz_serial_controller.py，将关节角数据以 json 格式发送至 Rviz 节点，从而实现对虚拟机械臂的控制。

1.3. Rviz 平台上的 OpenArm 仿真验证：

1.3.1. DLS 伪逆法简介

作为重构后纯 C 语言控制核心的首次测试，我首先在 ROS2 工作空间中引入了和 AM-D02 构型相同的虚拟 OpenArm 七自由度机械臂模型进行闭环验证。

在这里，我使用了 DLS 伪逆法来处理逆运动学求解，通过引入一个阻尼系数 λ ，解决奇异性和不稳定性问题。流程是在每个迭代步骤中，计算关节的增量(dq)，

并更新当前的关节角度，通过正运动学的验证与反馈不断逼近，直到达到目标位置，是冗余机械臂和高精度轨迹跟踪场景的主流解法。

$$J^+ = J^T(JJ^T + \lambda^2 I)^{-1}$$

图 1.3.1.1 阻尼最小二乘法雅可比矩阵伪逆公式

如下表所示，我为机械臂设计了七种运动模式，分别用于验证正运动学，逆运动学，测试求解器性能等。

模式ID	类型	名称	描述	作用
模式 0	正运动学	关节角正弦波	各关节独立正弦波运动	验证硬件在环是成功
模式 1	正运动学	步进测试	5个固定位置循环切换	验证正运动学精度
模式 2	正运动学	慢速平滑正弦波	低频正弦波，运动更平滑	验证正运动学动态性能
模式 3	逆运动学	末端画圆	XY平面上的圆形轨迹，Z固定	验证平面轨迹跟踪精度
模式 4	逆运动学	末端X轴往返	X方向直线往复运动	验证单轴动态性能
模式 5	逆运动学	末端Y轴往返	Y方向直线往复运动	验证单轴动态性能
模式 6	逆运动学	末端Z轴往返	Z方向直线往复运动	验证垂直方向性能

图 1.3.1.2 七种运动学模式

1.3.2. OpenArm 末端轨迹规划

为了实现更加优雅与精准的控制，我选择使用 6*7 全量位姿雅可比进行逆运动学的求解。相较于仅表征末端线速度的 3*7 位置雅可比，6*7 位姿雅可比整合了线速度与角速度映射关系，具有同时管控位置 + 姿态、支持全维度运动约束、适配完整位姿任务、可统一做奇异点分析与冗余优化的核心优势，是工业机械臂轨迹控制、避障、关节优化的标准方案。

其标准形式如下所示：

$$\begin{bmatrix} \dot{p} \\ \omega \end{bmatrix} = J\dot{q}, \quad \begin{bmatrix} \dot{p} \\ \omega \end{bmatrix} \in \mathbb{R}^{6 \times 1}$$

注：前 3 行为位置雅可比；后 3 行为姿态雅可比。

我对其进行了画圆与三轴直线任务测试，逆运动学控制效果如下：

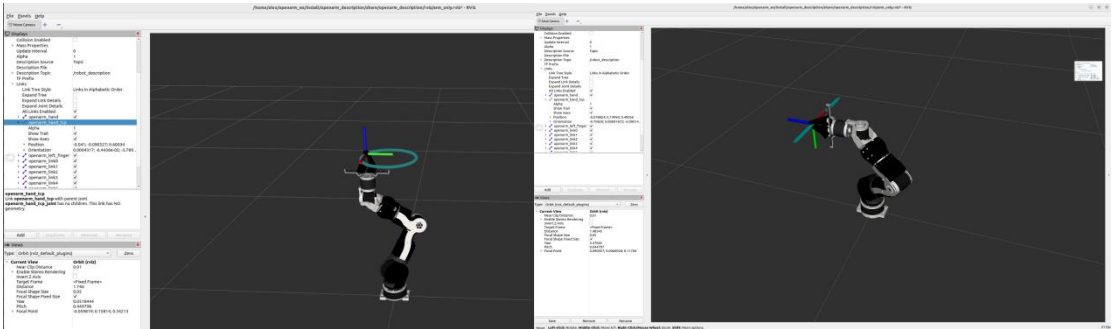


图 1.3.2.1 case3:XY 平面画圆

图 1.3.2.2 case4:沿 X 轴画直线

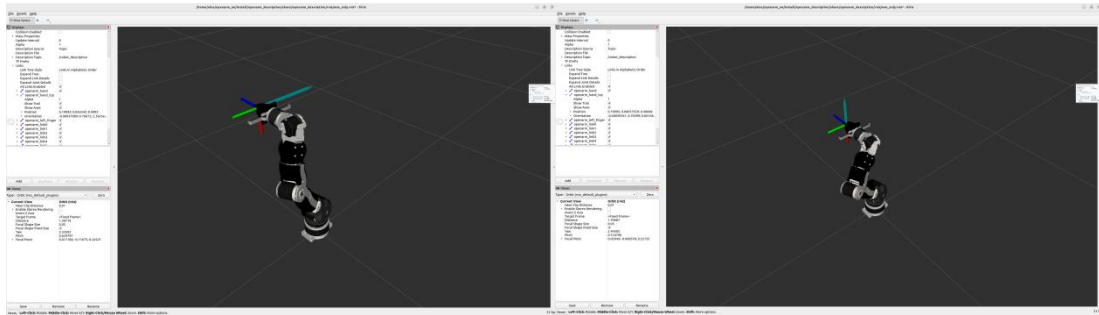


图 1.3.2.3 case5:沿 Y 轴画直线

图 1.3.2.4 case6:沿 Z 轴画直线

通过对比可以观察到,在 6×7 位姿版本的控制下,机械臂能在保证位置 + 姿态同时达标的前提下,利用剩余冗余自由度做二次优化,使整体的轨迹规划更加自然协调。

至此,我实现了单个 S-R-S 机械臂的完整运动学解算与轨迹规划。

1.4. AM-D02 单臂部署:

在完成了基于虚拟 OpenArm 的运动学算法基准测试后,项目进入了向 AM-D02 七自由度冗余机械臂进行部署适配的阶段。在此过程中,我将运动学求解器进行局部优化,使其更加适配 AM-D02 的硬件参数。本章重点阐述针对实体硬件进行的几何对齐、关节轴向校正以及致命姿态初始化缺陷的工程优化方案。

1.4.1. URDF 参数转写与关节轴校正

虚拟算法若要精准驱动真实硬件,其底层运动学链条的几何描述必须与实体的物理结构完全一致。

我在底层的嵌入式求解器中,定义了紧凑的 URDFJoint 静态结构体数组,将 AM-D02 的 URDF 描述文件中的 originxyz 和 rpy 平移与旋转偏置进行了像素级的硬编码转译。这保证了前向运动学计算中,连杆坐标系之间的齐次变换矩阵在空间长度和初始安装角度上与实物百分百贴合。

```

61 // =====
62 // 核心重构:严格提取自 AMDP URDF 文件(剔除固定连杆,只保留 7 个运动关节)
63 // =====
64
65 // 左臂 7 个运动关节的相对变换参数 (ArmL02_Joint 到 ArmL07Output_Joint)
66 static const URDFJoint urdf_joints_left[NUM_JOINTS] = {
67     {{ 0.050000f, 0.035202f, 0.000000f }, { -1.570796f, 0.000000f, 0.000000f }, { 0.0f, 0.0f, 1.0f }}, // ArmL02_Joint
68     {{ -0.032700f, 0.000000f, 0.085000f }, { 1.570796f, 0.000000f, -1.570796f }, { 0.0f, 0.0f, 1.0f }}, // AM-D02-J14_Joint
69     {{ -0.066250f, 0.000000f, -0.034100f }, { 0.000000f, 1.570800f, 0.000000f }, { 0.0f, 0.0f, -1.0f }}, // ArmL04_Joint
70     {{ -0.013422f, -0.031550f, -0.152890f }, { 1.570796f, 0.000000f, 0.000000f }, { 0.0f, 0.0f, 1.0f }}, // ArmL05_Joint
71     {{ -0.008410f, -0.094130f, -0.033199f }, { 1.570796f, 0.000000f, 0.000000f }, { 0.0f, 0.0f, 1.0f }}, // ArmL06_Joint
72     {{ -0.038200f, -0.000301f, 0.137000f }, { 3.141600f, 1.570800f, 0.000000f }, { 0.0f, 0.0f, -1.0f }}, // ArmL07_Joint
73     {{ 0.000000f, -0.020500f, -0.037500f }, { 1.570800f, 1.570800f, 0.000000f }, { 0.0f, 0.0f, 1.0f }}, // ArmL07Output_Joint
74 };
75
76 // 右臂 7 个运动关节的相对变换参数 (ArmR02_Joint 到 ArmR07Output_Joint)
77 static const URDFJoint urdf_joints_right[NUM_JOINTS] = {
78     {{ 0.050000f, -0.035202f, 0.000000f }, { 1.570800f, -1.570800f, 0.000000f }, { 0.0f, 0.0f, 1.0f }}, // ArmR02_Joint
79     {{ 0.000000f, 0.031700f, 0.085000f }, { -1.570800f, 0.000000f, 0.000000f }, { 0.0f, 0.0f, 1.0f }}, // AM-D02R-J03_Joint
80     {{ -0.066250f, 0.000000f, -0.034100f }, { 0.000000f, 1.570796f, 0.000000f }, { 0.0f, 0.0f, -1.0f }}, // ArmR04_Joint
81     {{ -0.013422f, -0.031550f, -0.152890f }, { 1.570800f, -1.570800f, 0.000000f }, { 0.0f, 0.0f, -1.0f }}, // ArmR05_Joint
82     {{ -0.094130f, 0.008410f, -0.033500f }, { 1.570796f, 0.000000f, -1.570796f }, { 0.0f, 0.0f, -1.0f }}, // ArmR06_Joint
83     {{ -0.038200f, 0.000000f, 0.138000f }, { 3.141600f, 1.570800f, 0.000000f }, { 0.0f, 0.0f, -1.0f }}, // ArmR07_Joint
84     {{ 0.000000f, 0.019500f, -0.038100f }, { 1.570800f, 1.570800f, 0.000000f }, { 0.0f, 0.0f, -1.0f }}, // ArmR07Output_Joint
85 };
86

```

图 1.4.1.1 URDF 参数对齐

项目对雅可比矩阵的列向量计算进行了彻底重构。对于前 6 个轴，严格对齐其基座及各级连杆的旋转切线。对于第 7 轴，通过直接从运动学结构体中动态提取真实局部轴向向量并利用当前连杆的累计旋转矩阵将其投影变换至统一的基坐标系（Base Link）下，完美补偿了其微小的倾斜耦合。这确保了逆运动学（IK）梯度下降迭代时，关节角的修正方向在空间中完全正确。

1.4.2. 平滑轨迹规划的实现

在前述运动学算法及物理几何参数完全对齐的基础上，系统虽已具备了离散目标点位的解算能力，但在真实的物理硬件上，若直接将不连续的任务指令下发给电机，会引发强烈的动力学冲击。为此，项目在嵌入式底层设计并实现了一套轻量级机载轨迹规划器。

在切换不同控制模式时，关节空间的目标角度往往存在阶跃。为了消除瞬间突变，我引入了一个耗时固定的“过渡状态”，通过五次多项式（Minimum Jerk）样条插值算法实现平滑平移。

五次多项式样条插值是一种高阶插值方法，它在每个区间上使用五次多项式，并且要求函数值、一阶导数、二阶导数、三阶导数和四阶导数在节点处连续，因此能够生成极其平滑的曲线，特别适用于机器人轨迹规划、精密机械运动控制等对运动平滑性要求极高的领域。

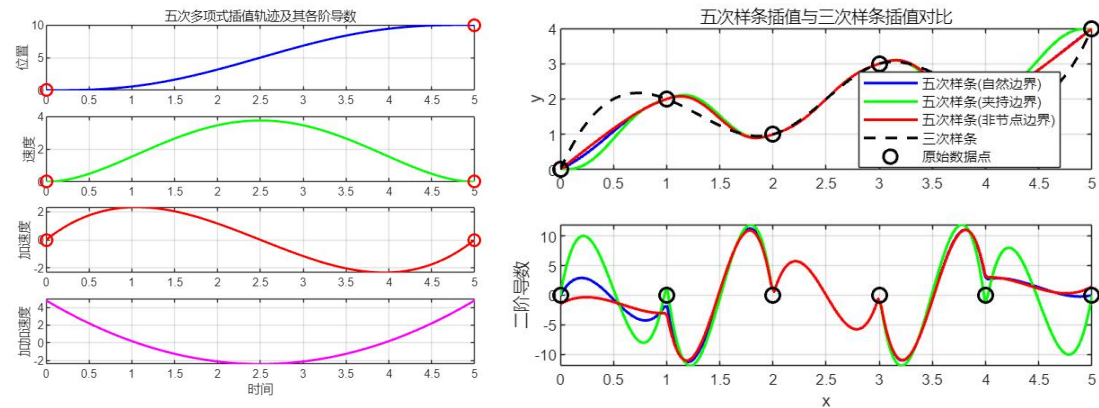


图 1.4.2.1 五次多项式样条插值 Matlab 仿真效果

当上位机发送模式切换指令时，底层状态机并不立刻执行新轨迹，而是触发过渡逻辑：

- 锁存起点：瞬间记忆当前 7 个关节的真实物理角度，作为插值起点 $q_{start,i}$ 。
- 设定终点：将无奇异的健康 L 形姿态作为插值终点 $q_{target,i}$ 。
- 时间对齐：启动过渡定时器，设定总过渡时间为 $T_{trans} = 2.0s$ 。

在控制周期内（每 20ms），利用归一化时间 $\tau = t/T_{trans}$ 计算无冲击曲线。五次多项式的边界条件设定为始末两端的速度、加速度均为 0，其无量纲缩放因子的核心公式为：

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5 \quad (\tau \in [0,1])$$

通过对 7 个关节分别进行独立的时域映射，生成每一帧的平滑目标角度：

$$q_i(t) = q_{start,i} + (q_{target,i} - q_{start,i}) \cdot s\left(\frac{t}{T_{trans}}\right)$$

该算法保证了关节空间在位置（ q ）、速度（ $\dot{q}$ ）、加速度（ $\ddot{q}$ ）等五阶连续，彻底消除了阶跃响应。

当过渡状态结束、机械臂正式进入工作模式时，规划器将切入**笛卡尔空间插值引擎**，以满足具身智能任务对末端均匀运动的要求。传统算法通常基于固定角度频率 $\theta = \omega t$ 驱动机械臂末端线速度，可能导致曲线半径改变时末端线速度发生突变。本项目中，我引入了恒定线速度驱动机制，基于期望线速度 v_{linear} 的非均匀时间步机制。以画圆轨迹为例，规划器首先根据圆半径 R 动态计算跑完一圈所需的总周期 T_{total} ：

$$T_{total} = \frac{2\pi R}{v_{linear}}$$

在每个 20ms 周期内，利用相对时间时钟求取当前平滑相位角：

$$\theta(t) = 2\pi \cdot fmod\left(\frac{t}{T_{total}}, 1.0\right)$$

得出轨迹公式输出：

$$\begin{cases} X_{target}(t) = X_{center} \\ Y_{target}(t) = Y_{center} + R \cdot \sin(\theta(t)) \\ Z_{target}(t) = Z_{center} + R \cdot (1.0 - \cos(\theta(t))) \end{cases}$$

此外，我利用 $1.0 - \cos(\theta)$ 的数学特性，保证了 $t=0$ 启动瞬间目标位置的变化率从 0 开始平滑爬升，实现了真正意义上的低冲击冷启动。

1.4.3. AM-D02 轨迹规划效果

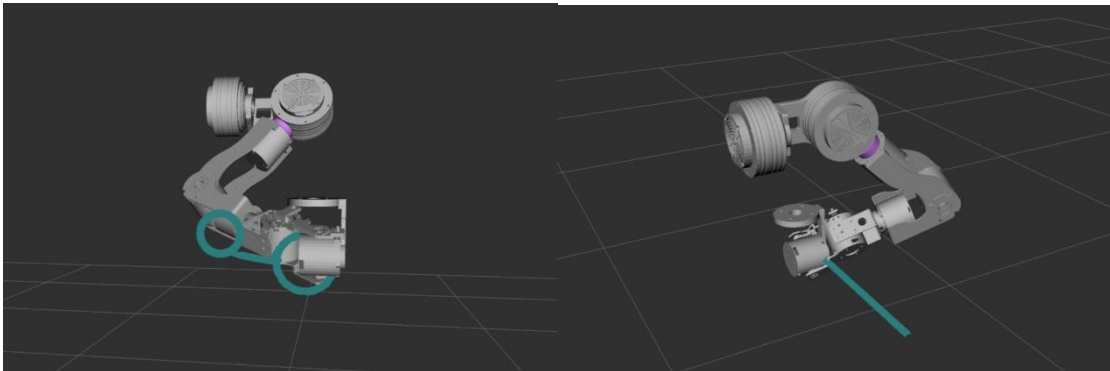


图 1.4.3.1 case3:XY 平面画圆

图 1.4.3.2 case4:沿 X 轴画直线

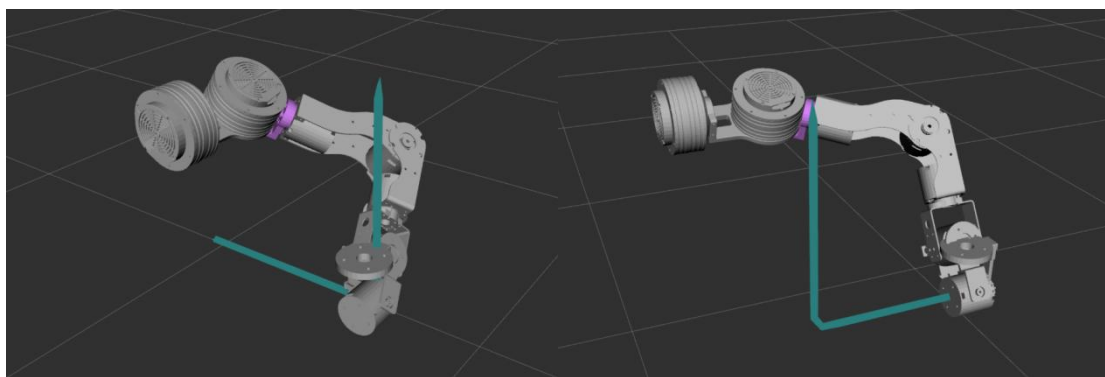


图 1.4.3.3 case5:沿 Y 轴画直线

图 1.4.3.4 case6:沿 Z 轴画直线

1.5. AM-DP 双臂解耦与扩展：

在成功完成 AM-D02 单臂的嵌入式运动学代码编写及底层控制验证后，为了进一步提升机器人的作业能力与双臂协同作业水平，需要将现有的控制架构扩展为 AM-DP 双臂机器人总装控制系统。

1.5.1. 核心转化策略与优势

在单臂向双臂架构扩展的过程中，本项目摒弃了将双臂合并为 14 自由度巨型矩阵的传统做法，而是采用了“动力学解耦、独立求解器路由”的先进控制架构。

系统在底层实例化了两个完全独立的 7 自由度运动学求解器实例：

left_solver 和 right_solver。在每一帧控制周期内，主控分别采集和生成左、右臂的笛卡尔空间目标位姿，随后串行触发两次 6×7 雅可比矩阵及 DLS 伪逆的迭代计算，最后将解算出的双臂共 14 个运动关节角度进行 JSON 格式的串行打包。

这种方案的核心优势是能够在极低的计算开销下实现高频控制：STM32F407 拥有硬件浮点运算单元（FPU），在其 168MHz 主频下，单个 7 轴求解器单次迭代及解算可在 $500 \mu s$ 内完成。如果强行建立 14 自由度的全维求解器，矩阵乘法与求逆的计算开销将呈立方级 $O(N^3)$ 暴涨，极易导致嵌入式主控芯片由于算力耗尽而引发系统死锁或主循环频率暴跌。采用双求解器串行解算，可在 1.5ms 内轻松完成双臂实时逆解，稳稳锁定 50Hz 以及更高的控制外环频率。

此外，左、右臂的底层解算核心算法在代码层面实现了高度的模块化复用。通过在结构体中引入 arm_type 标志位，实现了几何数据源的动态路由，不仅极大精简了嵌入式 Flash 的代码体积，还显著提升了系统的可维护性。

1.5.2. 双臂自碰撞检测

自碰撞检测的核心思路是“全车坐标系统一”与“三维几何体简化”，在嵌入式主控中以极低的计算开销评估空间干涉：

为了实现坐标系统一,要利用固定的安装几何参数,将左臂和右臂通过正向运动学(FK)解算出的各关节本地矩阵,实时投影到全车统一的世界参考坐标系——车体躯干坐标系(Body0422_Link)下。

由于模型的具体网格过于复杂,无法使用嵌入式系统实时计算,因此拓扑几何简化十分必要。我将AM-DP的主要部分简化为以下三种简单模型:

- **胶囊体(Capsule)**: 将双臂的机械连杆(大臂、小臂)简化为由“空间线段 + 包围半径 + 安全裕量”组成的胶囊体。
- **包围球(Sphere)**: 将末端工具(手掌)简化为“中心点 + 半径”的球体。
- **轴齐包围盒(AABB Box)**: 将复杂的车体躯干简化为一个由 x, y, z 绝对边界定义的立方体盒子。



图 1.5.2.1 自碰撞检测机制

在数学判别算法中,我将所有可能发生的碰撞归纳为三种类型,并分别对其设置判断条件。

- **臂与臂碰撞**: 采用线段间距离算法(Lumelsky 算法),计算空间中任意两条关节线段之间的最短距离 D_{min} 。若 $D_{min} < R_1 + R_2 + \text{安全裕量}$,则判定碰撞。
- **手碰手/自身折叠**: 计算两球心(如左手与右手、手掌与同侧肩膀)之间的空间欧氏距离是否小于半径之和。
- **部件撞击身体**: 通过标量范围比对,严格检查手腕及手掌末端点的空间坐标是否侵入了 AABB 车体包围盒的边界。

1.5.3. AM-DP 轨迹规划效果

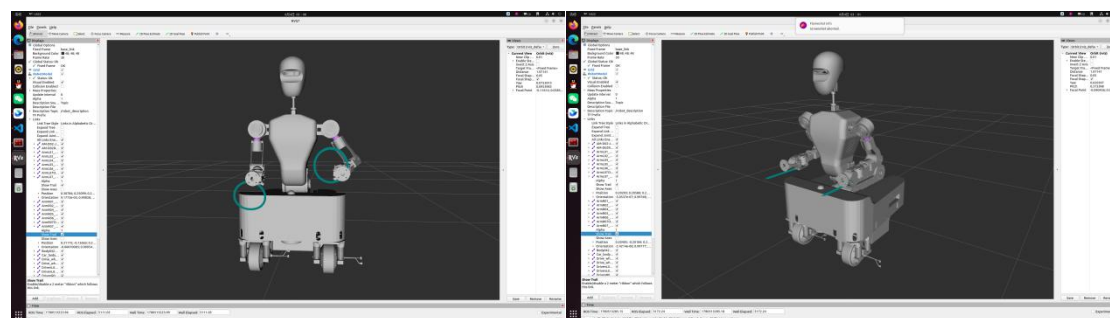


图 1.5.3.1 case3:XY 平面画圆

图 1.5.3.2 case4:沿 X 轴画直线

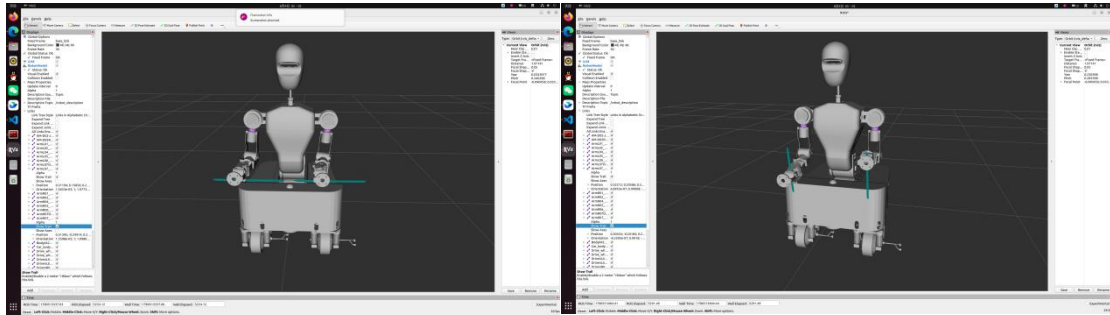


图 1.5.3.3 case5:沿 Y 轴画直线

图 1.5.3.4 case6:沿 Z 轴画直线

1.6. 动力学建模与控制：

在前述的高效运动学解算器与平滑轨迹规划器的基础上，系统已能生成连续、满秩的关节空间理想状态。然而，真实物理硬件（AM-D02 机械臂与 AM-DP 半人形机器人）具有质量、转动惯量、以及传动摩擦等强烈的非线性动力学特征。

如果控制系统仅停留在运动学层面（仅给定位姿指令），机械臂在高速运动或承载具身抓取任务时，会因为惯性滞后产生严重的轨迹跟踪偏差。为此，本项目在运动学核心之上，自主重构了嵌入式递归牛顿-欧拉动力学求解器，并接入达妙关节伺服电机的底层控制总线，实现了融合动力学前馈的 MIT 控制与串级 PID 混合架构，成功跨越了 Sim2Real 的动力学鸿沟。

1.6.1. 动力学系统架构

首先，项目基于 Mujoco 实现了动力学仿真系统，系统采用四层分层架构，按职责与运行环境划分：



图 1.6.1.1 动力学四层仿真架构

动力学仿真系统设置了三种模式，分别为软件在环仿真模式，蒙特卡洛工作空间分析模式，真机控制模式，具体接口与说明如下表：

模式	入口	说明
sim	./run.sh sim	Python MuJoCo 物理仿真+C 实时控制回路
real	./run.sh real left/right/both	SocketCAN 真机 MIT 力矩控制
mc	./run.sh mc	蒙特卡洛关节空间采样 → 末端工作空间 范围分析

仿真基于 MuJoCo 物理引擎，模型从 URDF 构建并通过 scene.py 增强（添加 mocap 目标显示体、TCP 标记、坐标轴可视化）。

MuJoCo（Multi-Joint dynamics with Contact）是 Google DeepMind 开源的高精度多体动力学仿真引擎，主打接触丰富场景（碰撞、摩擦、抓取、装配）的稳定、快速、可微物理模拟。

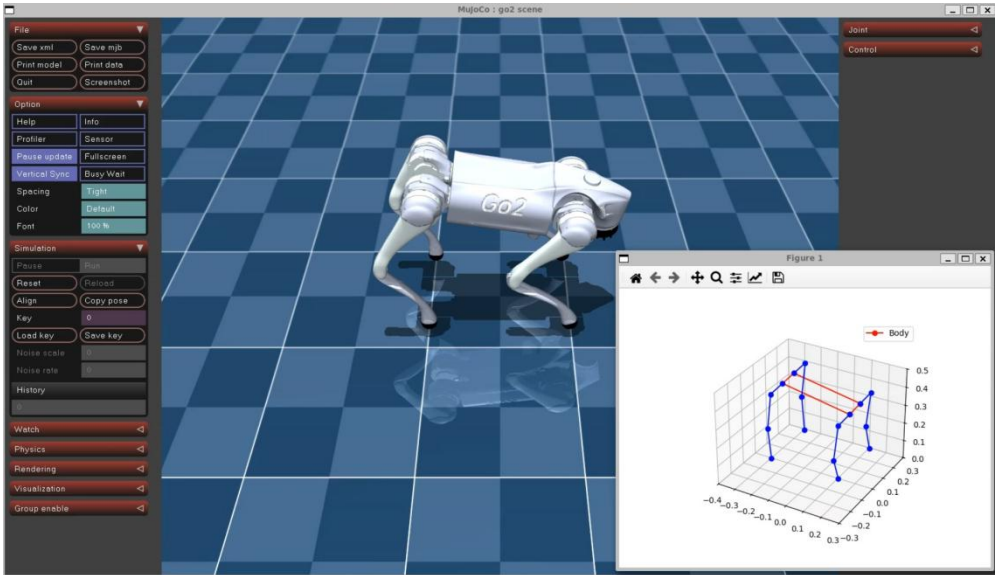


图 1.6.1.2 Mujoco 界面演示

UDP（用户数据报协议）是 TCP/IP 协议簇里无连接、面向数据报的传输层协议，是轻量、低延迟、无连接的数据报协议，不保证可靠，但实时性极强，是机械臂、机器人实时运动控制的主流网络方案。

本项目中，C 控制器与 Python 仿真服务器通过 UDP 在 localhost:9876 通信。协议为请求-响应模式：

- C 端发送 “INIT” → Python 端返回初始状态包
- 循环中：C 端发送 tau[14]→Python 端步进仿真→返回 45×double 状态包
- 超时 2 秒无响应视为断联

此外，本项目使用 Rerun 作为机械臂的数据监控仪表盘 + 实时示波器。Rerun 是一个用于机器人和计算机视觉的开源可视化 SDK（rerun.io）。

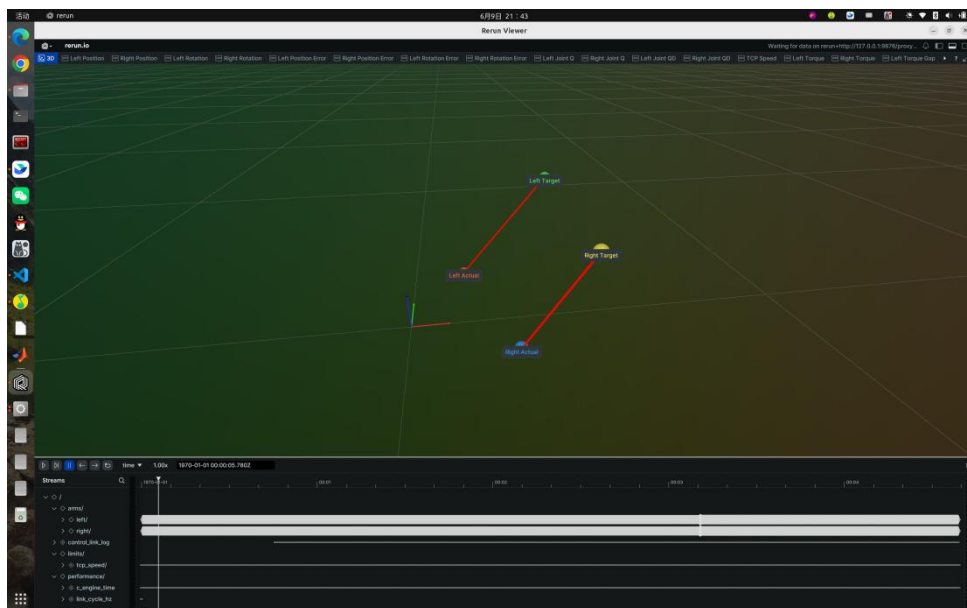


图 1.6.1.2 Rerun 界面演示

在这个双臂机械臂工程中，它用来在仿真和真机运行时可视化所有关键控制数据，调试控制器时，不需要对着终端打印的数字猜发生了什么——打开 Rerun 面板就能同时看到：

- 14 个关节的力矩曲线
- 末端 TCP 在 3D 空间中的移动轨迹
- 跟踪误差随时间的变化
- 安全边界是否被逼近

这对调参（笛卡尔 PD 增益、S 曲线速度、摩擦力参数等）和排查异常（力矩饱和、关节限位、通信丢帧）非常有价值。

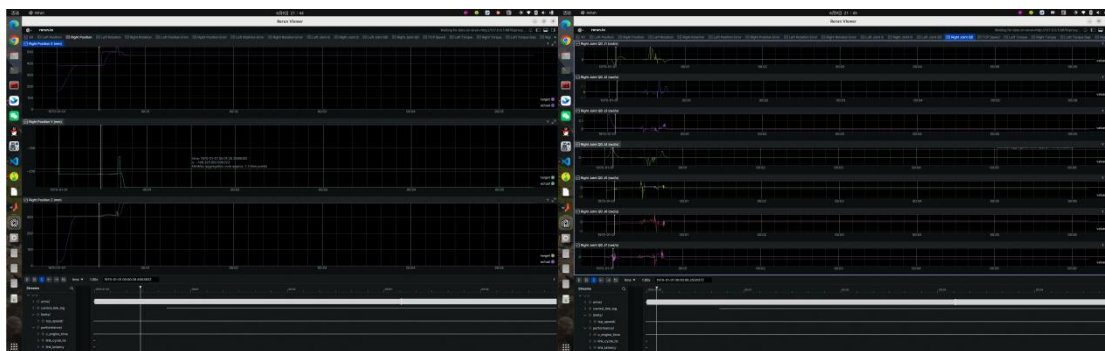


图 1.6.1.3 右臂末端位置曲线

图 1.6.1.4 右臂末端速度曲线

1.6.2. 刚体动力学建模

C 控制核心自实现了一个轻量刚体动力学库(RBDL-Lite)，采用 Featherstone 空间向量代数。每个连杆的动力学参数：

参数	符号	维度	说明
质量	m_i	标量	kg
质心矩	$\mathbf{h}_i = m_i \cdot \mathbf{c}_i$	3	质量 × 质心坐标
原点惯性张量	$\mathbf{I}_i$	6	已通过平行轴定理平移至关节原点
父→子变换	$\mathbf{E}_p, \mathbf{r}_p$	9+3	$\mathbf{q}=0$ 时的旋转矩阵与平移
运动子空间	$\mathbf{S}_i$	6	旋转关节 = [axis; 0,0,0]

惯性参数来源：从 SolidWorks URDF 导出，经平行轴定理将质心惯性张量转换为关节原点惯性张量：

$$\mathbf{I}_{origin} = \mathbf{I}_{com} + m \cdot [(\mathbf{c}^T \mathbf{c})\mathbf{E} - \mathbf{c}\mathbf{c}^T]$$

左右臂各 7 个连杆的惯性参数独立存储 (model_lib.c)，包括质量、质心、惯性张量 (I_{xx} , I_{xy} , I_{xz} , I_{yy} , I_{yz} , I_{zz})。

左臂连杆惯性参数示例：

连杆	质量 (kg)	质心 (m)	转动惯量 I (kg·m ²)
Link 1	1.1012	(-0.0036, 0.0000, 0.0792)	0.001423
Link 2	0.0804	(-0.0431, 0.0000, -0.0168)	3.44e-5
Link 3	1.1191	(-0.0071, -0.0003, -0.0837)	0.001218
Link 4	0.4800	(-0.0078, -0.0616, -0.0327)	0.000385
Link 5	0.6140	(-0.0017, 0.0004, 0.0667)	0.000297
Link 6	0.3530	(0.0000, -0.0006, -0.0369)	0.000125
Link 7	0.1301	(0.0000, 0.0531, -0.0152)	5.94e-5

1.6.3. 基于递归牛顿-欧拉法（RNEA）的逆动力学求解器

为了在 STM32 微控制器上实现即时动力学计算，项目放弃了计算繁琐的拉格朗日方程法，采用计算复杂度同样为 $\mathcal{O}(N)$ 且天然适合计算机循环迭代的递归牛顿-欧拉算法（Recursive Newton-Euler Algorithm, RNEA）。动力学求解器在每个 20ms 控制周期内执行“前向”与“后向”两次递归迭代，直接计算维持当前运动状态所需的理论关节驱动力矩 τ_{ff} 。其具体递归迭代算法如下：

动力学前向递归：

已知当前各关节的真实/目标角度 q_i 、角速度 $\dot{q}_i$ 及角加速度 $\ddot{q}_i$ ，向外递推计算每个连杆质心的线加速度与角加速度。考虑基座重力补偿，设定基座初始线加速度 $\dot{\mathbf{v}}_0 = [0, 0, -9.81]^T m/s^2$ 。

第 i 连杆的角速度与角加速度递推公式为：

$$\boldsymbol{\omega}_i = R_i^{i-1} \boldsymbol{\omega}_{i-1} + \mathbf{z}_0 \dot{q}_i$$

$$\dot{\boldsymbol{\omega}}_i = R_i^{i-1} \dot{\boldsymbol{\omega}}_{i-1} + R_i^{i-1} \boldsymbol{\omega}_{i-1} \times \mathbf{z}_0 \dot{q}_i + \mathbf{z}_0 \ddot{q}_i$$

第 i 连杆关节起点及质心 (CoM) 的线加速度递推公式为：

$$\dot{\mathbf{v}}_i = R_i^{i-1} \dot{\mathbf{v}}_{i-1} + \dot{\boldsymbol{\omega}}_i \times \mathbf{p}_i + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \mathbf{p}_i)$$

$$\dot{\mathbf{v}}_{C,i} = \dot{\mathbf{v}}_i + \dot{\boldsymbol{\omega}}_i \times \mathbf{r}_{C,i} + \boldsymbol{\omega}_i \times (\boldsymbol{\omega}_i \times \mathbf{r}_{C,i})$$

其中, $\mathbf{p}_i$ 为连杆 $i-1$ 到连杆 i 的坐标系位移, $\mathbf{r}_{C,i}$ 为连杆 i 坐标系原点到其质心的距离, 均由 URDF 物理参数文件提供静态常数支持。

动力学后向递归：

利用牛顿第二定律和欧拉方程, 结合各连杆的静态质量 m_i 与质心转动惯量矩阵 $\mathbf{I}_i$, 向内递推求解关节间的作用力和力矩, 最终投影得到关节驱动力矩。

第 i 连杆受到的合外力和合外力矩为：

$$\mathbf{F}_i = m_i \dot{\mathbf{v}}_{C,i}$$

$$\mathbf{N}_i = \mathbf{I}_i \dot{\boldsymbol{\omega}}_i + \boldsymbol{\omega}_i \times (\mathbf{I}_i \boldsymbol{\omega}_i)$$

结合相邻连杆约束, 计算各关节受到的净力和净力矩：

$$\mathbf{f}_i = R_{i+1}^i \mathbf{f}_{i+1} + \mathbf{F}_i$$

$$\boldsymbol{\tau}_i = R_{i+1}^i \boldsymbol{\tau}_{i+1} + \mathbf{p}_i \times (R_{i+1}^i \mathbf{f}_{i+1}) + (\mathbf{p}_i + \mathbf{r}_{C,i}) \times \mathbf{F}_i + \mathbf{N}_i$$

最终, 提取旋转轴 $\mathbf{z}_0$ 方向的力矩分量, 作为动力学前馈力矩 $\tau_{ff,i}$ ：

$$\tau_{ff,i} = \boldsymbol{\tau}_i^T \mathbf{z}_0 + b_i \dot{q}_i + f_{c,i} \text{sign}(\dot{q}_i)$$

其中, b_i 和 $f_{c,i}$ 为实验室在实物调试中辨识出的关节粘性摩擦与库伦摩擦补偿系数。

1.6.4. 动力学控制算法设计

动力学系统控制架构如下：

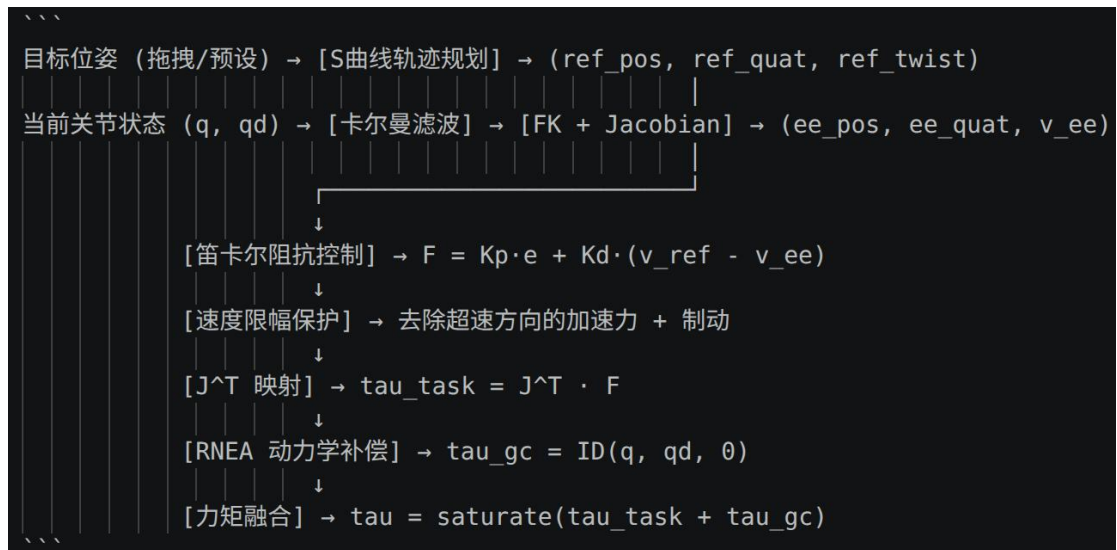


图 1.6.4.1 动力学控制算法架构

其中，笛卡尔空间阻抗控制就是在机器人末端直角坐标系（X/Y/Z + 姿态）里，把机械臂等效成一个可自由调软硬的“质量 - 弹簧 - 阻尼”系统，让末端受力后柔顺变形、力消失又回弹，从而安全、可控地和环境接触。

控制律在任务空间定义，产生六维力螺旋 $\mathbf{F} \in \mathbb{R}^6$ ：

$$\mathbf{F} = \mathbf{K}_p \cdot \mathbf{e} + \mathbf{K}_d \cdot (\mathbf{v}_{ref} - \mathbf{v}_{ee})$$

其中：

$$\mathbf{e} = \begin{bmatrix} \mathbf{p}_{ref} - \mathbf{p}_{ee} \\ \mathbf{e}_{rot} \end{bmatrix}, \mathbf{e}_{rot} = 2 \cdot \arctan2(\|\mathbf{q}_{err,v}\|, q_{err,w}) \cdot \frac{\mathbf{q}_{err,v}}{\|\mathbf{q}_{err,v}\|}$$

姿态误差通过四元数之差转换为轴角表示：

$$\mathbf{q}_{err} = \mathbf{q}_{target} * \mathbf{q}_{current}^{-1}, \mathbf{e}_{rot} = \text{AxisAngle}(\mathbf{q}_{err})$$

$$\mathbf{v}_{ee} = \mathbf{J}(q) \cdot \dot{q}$$

PD 增益矩阵：

轴	K_p	K_d	单位
X, Y, Z 平移	170.0	65.0	N/m, N·s/m
Roll, Pitch, Yaw 旋转	8.0	4.0	N·m/rad, N·m·s/rad

任务空间力通过雅可比转置映射到关节空间：

$$\boldsymbol{\tau}_{task} = \mathbf{J}^T(q) \cdot \mathbf{F}$$

随后再结合动力学前馈补偿，得出完整关节力矩输出：

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{task} + \boldsymbol{\tau}_{gc}(q, \dot{q})$$

其中 $\boldsymbol{\tau}_{gc}$ 由 RNEA($q, \dot{q}, 0$) 计算，补偿重力和科氏力/离心力。这使得 PD 控制只需处理误差修正，大幅减小跟踪误差和稳态力需求。

为防止 TCP 超速导致失控，控制核心包含主动速度限幅：

1. **检测**：计算 TCP 当前线速度 $v_{ee} = \|\mathbf{J}_v \cdot \dot{q}\|$
2. **限速阈值**： $v_{max} = 0.05 \text{ m/s}$
3. **保护动作**：超过阈值时，去除沿当前速度方向的加速力分量，并施加与超速量成比例的制动力：

$$\mathbf{F}_{brake} = \frac{K_{d,x} + K_{d,y} + K_{d,z}}{3} \cdot (v_{ee} - v_{max})$$

此外，为了应对传感器的噪声问题，实现更平滑低抖动的控制，每关节使用独立 1D 卡尔曼滤波器处理测量噪声。

卡尔曼滤波（Kalman Filter, KF）是一种线性、高斯噪声假设下的最优状态估计算法。专门用于从带噪声的传感器观测 + 系统动力学模型中，估计系统真实状态（位置、速度、角度、姿态等），广泛用于机器人、自动驾驶中。

完整的卡尔曼滤波可分为两大阶段：预测阶段（时间更新）+ 更新阶段（测量更新）。

预测阶段，利用上一时刻状态，预测当前状态与不确定性：

$$\hat{x}_k^- = \hat{x}_{k-1}, P_k^- = P_{k-1} + Q$$

更新阶段，使用传感器观测修正预测值，得到最优估计，并更新不确定性：

$$K_k = \frac{P_k^-}{P_k^- + R}, \hat{x}_k = \hat{x}_k^- + K_k(z_k - \hat{x}_k^-), P_k = (1 - K_k)P_k^-$$

此处我取过程噪声协方差 $Q = 0.001$ ，测量噪声协方差 $R = 0.1$ ，那么卡尔曼增益 K 会很小，意味着更信任预测，不太信任传感器测量，这样做能够很好的抑制传感器噪声带来的抖动。

1.7. Sim2Real 跨越：

1.7.1. 分层安全策略

系统采用三层级联安全保护：

层级	机制	触发条件	响应
关节限位	位置/速度检查	q 或 q_d 超出内收限位	进入 Safety Latch
力矩限幅	输出饱和	$ \tau_i > \text{力矩上限}$	钳位到 $\pm \text{limit}$
非有限检测	NaN/Inf 检查	任何传感器数据非有限	立即 Safety Latch

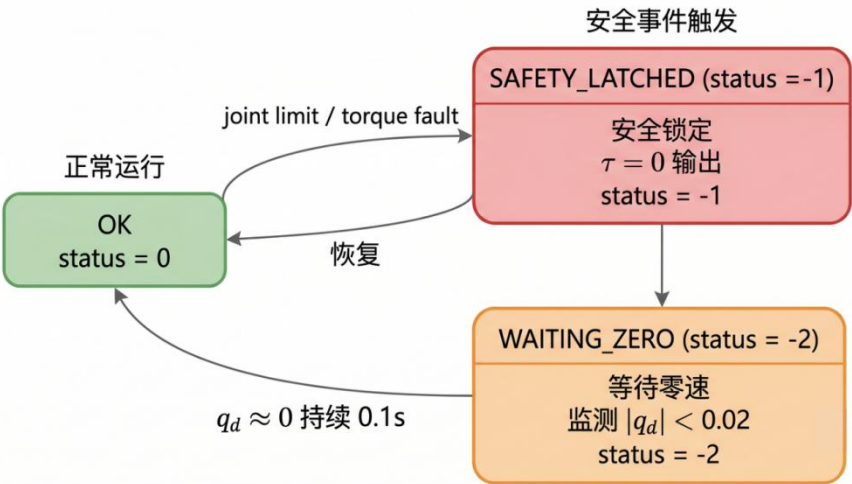


图 1.7.1.1 Safety Latch 状态机

- **触发条件：** 关节位置/速度超出安全限位（限位向内收缩 1%），或力矩超过关节极限，或检测到非有限值
- **激活响应：** 对应激活臂输出全零力矩
- **恢复条件：** 所有激活臂关节速度 $|\dot{q}_i| < 0.02 \text{ rad/s}$ ，持续 0.1 秒

1.7.2. 真机部署

AM-DP 机器人的硬件拓扑结构如下：

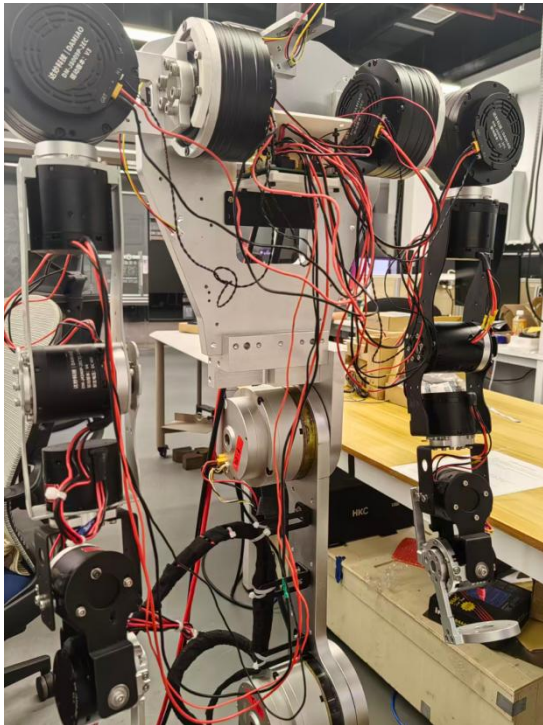


图 1.7.2.1 AM-DP 上半身实物

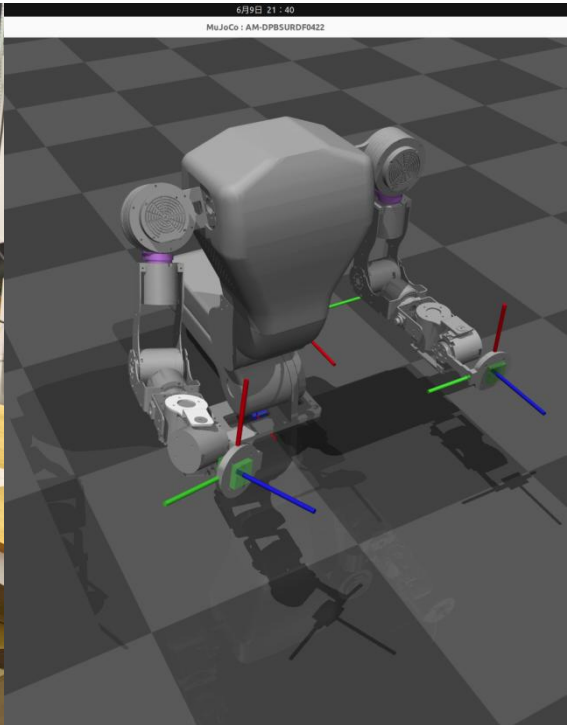


图 1.7.2.2 AM-DP (Mujoco 视角)

具体电机配置为单臂两个 DM8009，两个 DM4340 和 3 个 DM4310。

关节	电机型号	CAN ID (发送)	MST ID (接收反馈)
1, 2	DM8009	0x01, 0x02	0x11, 0x12
3, 4	DM4340	0x03, 0x04	0x13, 0x14
5, 6, 7	DM4310	0x05-0x07	0x15-0x17

SocketCAN 是 Linux 内核原生的 CAN 总线协议栈，核心是把 CAN 总线当作网络接口，用标准 Socket API 收发 CAN 帧，彻底屏蔽硬件差异。它能够把 CAN 总线变成 Linux 原生网络接口，统一、稳定、易开发、易调试，完美匹配机械臂分布式实时控制需求。本项目中 SocketCAN 配置如下：

参数	左臂	右臂
CAN 接口	can0	can1
波特率 (nominal)	1 Mbps	1 Mbps
数据波特率 (CAN-FD)	5 Mbps	5 Mbps
电机 ID	1..7	1..7

参数	左臂	右臂
q 映射	q[0..6]	q[7..13]
tau 映射	tau[0..6]	tau[7..13]

1.7.3. 蒙特卡洛工作空间分析

蒙特卡洛工作空间分析，是 Sim2Real 中用来对齐虚实模型、校准约束、预排风险、全域验证算法的基础工具。其主要原理为通过在各关节运动范围内随机生成海量关节角度样本，并利用正运动学求解对应末端位姿，最后统计、聚类、可视化所有可达点，得到工作空间云图 / 包络范围。核心作用就是让 仿真里能正常运动的机械臂，在实物上也能安全、一致地复现。

在 14 维关节限位空间内均匀随机采样 N 组关节角，通过正运动学计算左右臂末端 TCP 位姿，统计工作空间范围并构建 3D 凸包。

$$\mathbf{q}^{(k)} \sim U(\mathbf{q}_{min}, \mathbf{q}_{max}), k = 1, \dots, N$$

$$(\mathbf{p}_{ee}^{(k)}, \mathbf{q}_{ee}^{(k)}) = FK(\mathbf{q}^{(k)})$$

- 末端位置的 min / max / span / mean (x, y, z 三轴分量)
- 末端四元数的元素范围 (w, x, y, z)
- 3D 工作空间凸包 (ConvexHull, 通过 SciPy scipy.spatial.ConvexHull)
- 凸包内部最大轴对齐长方体 (AABB) — 即安全的 pos 目标输入范围

最大内部轴对齐长方体通过内点法 (Barrier Method) 优化：

$$\max_{\mathbf{c}, \mathbf{h}} \prod_{i=1}^3 h_i \quad \text{s.t.} \quad \mathbf{n}_j \cdot \mathbf{c} + |\mathbf{n}_j| \cdot \mathbf{h} + o_j \leq 0, \forall j$$

其中 $(\mathbf{n}_j, o_j)$ 为凸包半空间表示， $\mathbf{c}$ 为长方体中心， $\mathbf{h}$ 为半边长度。优化变量 $\mathbf{x} = [\mathbf{c}; \mathbf{h}] \in \mathbb{R}^6$ 。

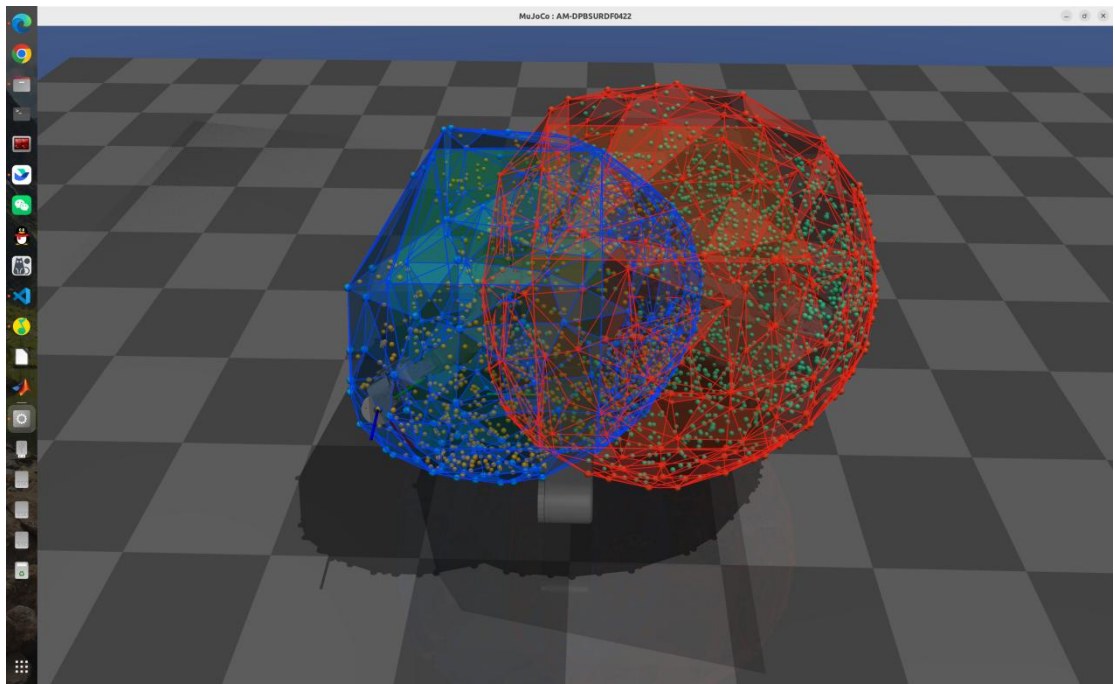


图 1.7.3.1 蒙特卡洛工作空间分析效果

1.7.4. 基于串级 PID 与 MIT 控制的 Sim2Real 动力学调优

为了解决虚拟仿真环境（完美刚体、无延迟）到物理硬件（有物理间隙、总线串行延迟、摩擦非线性）的 Sim2Real 鸿沟,项目在 AM-D02 机械臂与 AM-DP 半人形机器人实物上实施了动态串级 PID 与 MIT 前馈混合控制调优策略。

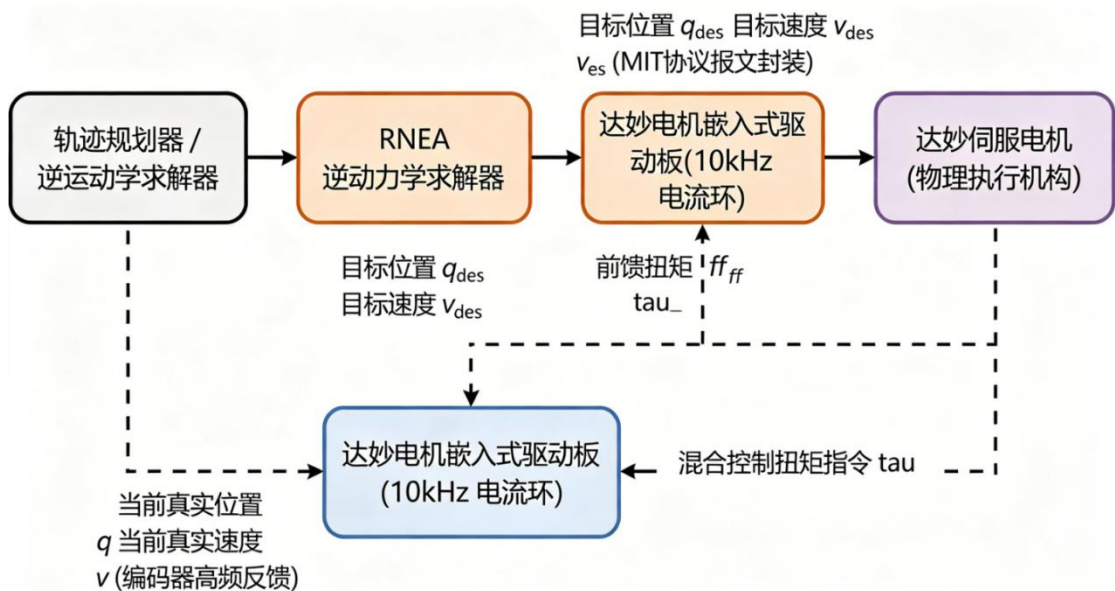


图 1.6.2.1 力位混控机制

项目采用了混合控制架构的设计。在低速/静态保持期（如刚开机、或过过渡期初始点），将电机的 K_p 适当调高， K_d 调低。此时控制系统表现为传统位置

外环-速度内环的串级 PID 控制形态。高位置增益确保了机械臂具有极强的静态刚度与抗负载扰动能力，能够精准停留在 L 形初始位置，消除了静摩擦导致的低频爬行。而在高速动态轨迹期（如连续画圆、直线往返运动期间），随着末端期望线速度 v_{linear} 爬升，系统动态调低电机驱动板内部的 K_p 增益，并增大 K_d 阻尼，将控制主导权完全移交给动力学前馈项 τ_{ff} 。

在 AM-DP 半人形机器人的双臂联合调试中，由于双臂并行的机械耦合振动，纯位置控制会引发强烈的系统共振。引入后向递归摩擦补偿模型后，通过精细微调达妙电机的 MIT 报文参数（将各个关节的 K_p 梯度式配置：肩部大电机 $K_p = 60.0$ ，腕部小电机 $K_p = 15.0$ ），成功消除了机械臂在高速换向瞬间的滞后现象。

实物跟踪数据表明，在融入了牛顿-欧拉动力学前馈后，AM-D02 末端在执行 0.03m/s 恒线速度连续画圆任务时，动态轨迹跟踪误差大幅降低。同时，彻底消除了模式切换瞬间因控制参数不匹配导致的关节抽搐与闪烁现象，完美实现了 Sim2Real 动力学层面的平滑跨越。

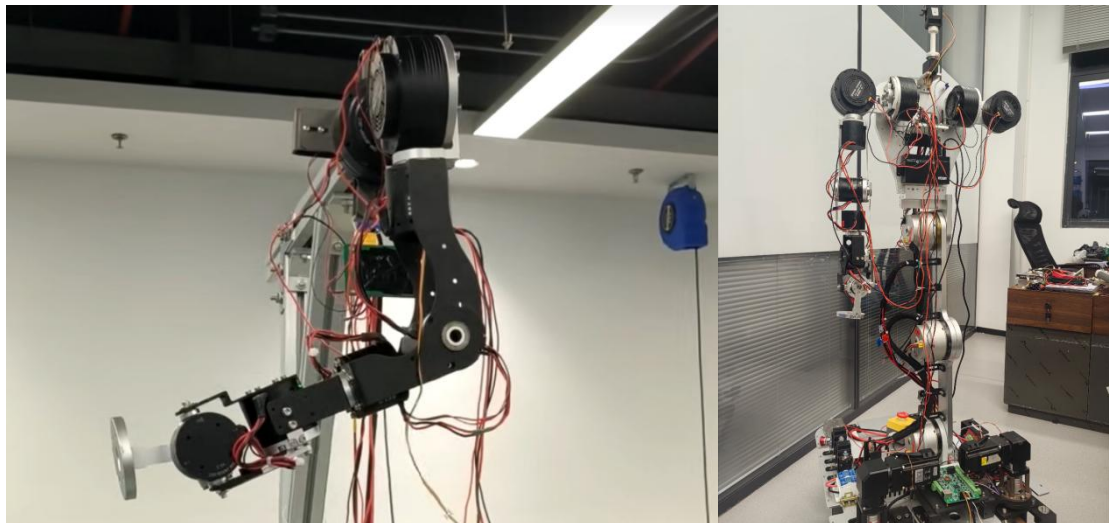


图 1.7.4.2 硬件实时控制